WebGL & three.js Cheat Sheet

WebGL 2 / GLSL ES 3.00 · three.js r187 · WebGPU · checked 1 November 2026 · Irurueta Docs, Graphics & Games

Versions and 2026 baseline

Item	Status
WebGL 1.0	= OpenGL ES 2.0, GLSL ES 1.00; last release 1.0.3
WebGL 2.0	= ES 3.0, GLSL ES 3.00 (`#version 300 es`); Chrome 56, Firefox 51, Edge 79, Safari/iOS 15; ~97 % of usage
OffscreenCanvas	Chrome 69, Firefox 105, Safari 17
three.js	**r187** (`three@0.187.0`, 2026-10-30); WebGL 1 dropped in r163
WebGPU	W3C draft, HTTPS only; Chrome/Edge 113, Android 121, Firefox 141 (Windows), Safari 26
Layer	ANGLE translates ES to D3D11, Metal, Vulkan or GL

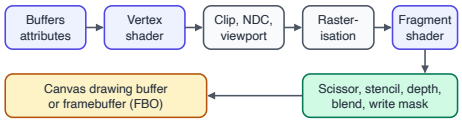
Context creation and attributes

```
const gl = canvas.getContext("webgl2", {
  alpha: false,           // opaque canvas
  antialias: true,        // MSAA, default fb only
  depth: true, stencil: false,
  premultipliedAlpha: true,
  preserveDrawingBuffer: false, // true costs speed
  powerPreference: "high-performance",
  failIfMajorPerformanceCaveat: false });
if (!gl) { /* try "webgl", else message */ }
canvas.addEventListener(
  "webglcontextcreationerror", e =>
  console.log(e.statusMessage));
```

Minimal draw recipe

```
// 1 shader  2 program
const vs = gl.createShader(gl.VERTEX_SHADER);
gl.shaderSource(vs, src); gl.compileShader(vs);
// check COMPILE_STATUS, getShaderInfoLog
const p = gl.createProgram();
gl.attachShader(p, vs); gl.attachShader(p, fs);
gl.linkProgram(p); // LINK_STATUS, getProgramInfoLog
// 3 buffer  4 VAO
const vao = gl.createVertexArray();
gl.bindVertexArray(vao);
const b = gl.createBuffer();
gl.bindBuffer(gl.ARRAY_BUFFER, b);
gl.bufferData(gl.ARRAY_BUFFER, data, gl.STATIC_DRAW);
gl.enableVertexAttribArray(0);
gl.vertexAttribPointer(0, 3, gl.FLOAT, false, 20, 0);
// 5 uniforms  6 draw
gl.useProgram(p);
gl.uniformMatrix4fv(loc, false, mvp);
gl.drawArrays(gl.TRIANGLES, 0, 3);  // or drawElements
```

Pipeline at a glance



GLSL ES 1.00 vs 3.00 and precision

1.00 (WebGL 1)	3.00 (WebGL 2)
no version line	`#version 300 es` first line
`attribute` (VS)	`in` + `layout(location=0)`
`varying`	`out` (VS), `in` (FS); `flat`, `smooth`
`gl_FragColor`	`out vec4 color;` (`layout(location=n)` for MRT)
`texture2D`, `textureCube`	`texture`, `texelFetch`, `textureSize`
uniforms only	`layout(std140) uniform Block {...};`

Fragment shaders need `precision mediump float;` (no default float). `highp` in a fragment shader is optional in 1.00, always present in 3.00. Query with `gl.getShaderPrecisionFormat`.

State order: viewport, depth, blend, cull

```
gl.viewport(0, 0, canvas.width, canvas.height);
gl.enable(gl.DEPTH_TEST); gl.depthFunc(gl.LEQUAL);
gl.depthMask(true);         // false for transparents
gl.enable(gl.CULL_FACE);
gl.cullFace(gl.BACK); gl.frontFace(gl.CCW);
gl.enable(gl.BLEND);        // straight alpha:
gl.blendFunc(gl.SRC_ALPHA, gl.ONE_MINUS_SRC_ALPHA);
// premultiplied: ONE, ONE_MINUS_SRC_ALPHA
gl.enable(gl.SCISSOR_TEST); gl.scissor(x, y, w, h);
gl.polygonOffset(1, 1);     // with POLYGON_OFFSET_FILL
gl.clearColor(0,0,0,1);
gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
```

Clip space is -w..w, NDC -1..1 (z too), window origin bottom-left; depth range `gl.depthRange`.

Textures: formats, storage, sampling

Internal format	Use
`RGBA8`	colour data (normals, masks)
`SRGB8_ALPHA8`	colour images, decoded to linear on sampling
`RGBA16F`, `R11F_G11F_B10F`	HDR targets (need `EXT_color_buffer_float` to render)
`DEPTH_COMPONENT24`	shadow maps (`COMPARE_REF_TO_TEXTURE`)

```
gl.pixelStorei(gl.UNPACK_FLIP_Y_WEBGL, true);
gl.texStorage2D(gl.TEXTURE_2D, levels, gl.SRGB8_ALPHA8,
w, h);
gl.texSubImage2D(gl.TEXTURE_2D, 0, 0, 0, w, h,
  gl.RGBA, gl.UNSIGNED_BYTE, imageBitmap);
gl.generateMipmap(gl.TEXTURE_2D);
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER,
  gl.LINEAR_MIPMAP_LINEAR);   // MAG_FILTER: LINEAR
// WRAP_S/T: CLAMP_TO_EDGE, REPEAT, MIRRORED_REPEAT
img.crossOrigin = "anonymous"; // + CORS headers
```

WebGL 1: non-power-of-two textures need CLAMP and no mipmaps. `texStorage` is immutable (WebGL 2).

FBO, MRT and MSAA

```
const fbo = gl.createFramebuffer();
gl.bindFramebuffer(gl.FRAMEBUFFER, fbo);
gl.framebufferTexture2D(gl.FRAMEBUFFER,
  gl.COLOR_ATTACHMENT0, gl.TEXTURE_2D, tex, 0);
gl.framebufferRenderbuffer(gl.FRAMEBUFFER,
  gl.DEPTH_ATTACHMENT, gl.RENDERBUFFER, rbo);
gl.drawBuffers([gl.COLOR_ATTACHMENT0,
                gl.COLOR_ATTACHMENT1]);   // MRT
// gl.checkFramebufferStatus === FRAMEBUFFER_COMPLETE
// MSAA: renderbufferStorageMultisample(.., 4, ..)
gl.bindFramebuffer(gl.READ_FRAMEBUFFER, msaa);
gl.bindFramebuffer(gl.DRAW_FRAMEBUFFER, resolve);
gl.blitFramebuffer(0,0,w,h, 0,0,w,h,
  gl.COLOR_BUFFER_BIT, gl.NEAREST);
gl.invalidateFramebuffer(gl.READ_FRAMEBUFFER,
  [gl.COLOR_ATTACHMENT0]);
```

The twelve most useful extensions

Extension	Gives you
`EXT_color_buffer_float`	render to RGBA32F, RGBA16F
`EXT_color_buffer_half_float`	render to half float (WebGL 1)
`OES_texture_float_linear`	linear filtering of float textures
`EXT_texture_filter_anisotropic`	anisotropic filtering
`WEBGL_compressed_texture_astc`	ASTC (mobile, Apple)
`WEBGL_compressed_texture_s3tc`	BC1 to BC3 (desktop)
`WEBGL_compressed_texture_etc`	ETC2 (Android)
`KHR_parallel_shader_compile`	non-blocking compile status
`EXT_disjoint_timer_query_webgl2`	GPU timing
`WEBGL_multi_draw`	many draws in one call
`OVR_multiview2`	one pass, several views (XR)
`WEBGL_debug_renderer_info`	GPU name (fingerprinting risk)

```
const ext = gl.getExtension("NAME"); if (!ext) fallback();
```

Also `WEBGL_lose_context` for tests.

Context-loss handler skeleton

```
canvas.addEventListener("webglcontextlost", e => {
  e.preventDefault();       // required to get restored
  cancelAnimationFrame(raf);
}, false);
canvas.addEventListener("webglcontextrestored", () => {
  getExtensions();          // re-acquire
  registry.rebuildAll();    // shaders, buffers,
  start();                  // textures, FBOs, state
}, false);
// test: gl.getExtension("WEBGL_lose_context")
//   .loseContext(); .restoreContext();
```

Ten-item performance checklist

1. Batch: fewer draw calls (instancing, merging, `BatchedMesh`)
2. Sort to cut program, texture and VAO switches
3. Never call `getError`, `readPixels` or `getParameter` per frame
4. Async readback: pixel pack buffer plus `fenceSync`
5. Compile with `KHR_parallel_shader_compile`
6. Cap `devicePixelRatio` (2); resize with `ResizeObserver`
7. `texStorage`, mipmaps, compressed textures (VRAM)
8. Update buffers with `bufferSubData`; no per-frame allocation
9. Render on demand when nothing changes
10. Measure: timer queries, Spector.js, devtools

three.js skeleton (r187)

```
<script type="importmap">{ "imports": {
 "three": ".../three@0.187.0/build/three.module.js",
 "three/addons/": ".../three@0.187.0/examples/jsm/" }}
</script>
import * as THREE from "three";
const renderer = new
THREE.WebGLRenderer({antialias:true});
renderer.setPixelRatio(Math.min(devicePixelRatio, 2));
document.body.append(renderer.domElement);
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(
  45, innerWidth / innerHeight, 0.1, 100);
function resize() {
  renderer.setSize(innerWidth, innerHeight);
  camera.aspect = innerWidth / innerHeight;
  camera.updateProjectionMatrix(); }
addEventListener("resize", resize); resize();
const timer = new THREE.Timer();
renderer.setAnimationLoop(t => {
  timer.update(t); const dt = timer.getDelta();
  renderer.render(scene, camera); });
```

Core three.js classes

Group	Classes
Scene	`Scene`, `Object3D`, `Group`, `Mesh`, `InstancedMesh`, `BatchedMesh`, `Points`, `Line`
Camera	`PerspectiveCamera`, `OrthographicCamera`
Geometry	`BufferGeometry`, `BoxGeometry`, `PlaneGeometry`, `SphereGeometry`
Material	`MeshStandardMaterial`, `MeshPhysicalMaterial`, `ShaderMaterial`
Light	`AmbientLight`, `DirectionalLight`, `PointLight`, `SpotLight`, `RectAreaLight`
Input	`Raycaster`, `OrbitControls`
Animation	`AnimationMixer`, `AnimationClip`, `AnimationAction`, `Timer`
Cleanup	`geometry.dispose()`, `material.dispose()`, `texture.dispose()`, `renderer.info`

Colour management defaults

```
THREE.ColorManagement.enabled = true;  // default
renderer.outputColorSpace =
  THREE.SRGBColorSpace;                // default
map.colorSpace = THREE.SRGBColorSpace; // colour maps
// normal, roughness, metalness, AO: NoColorSpace
renderer.toneMapping = THREE.ACESFilmicToneMapping;
renderer.toneMappingExposure = 1;      // default 1
// lights: candela, lux; decay = 2 (physical)
```

Colours given as hex are sRGB and converted to the linear working space. Vertex colours are linear.

glTF loading with Draco and KTX2

```
import { GLTFLoader } from
"three/addons/loaders/GLTFLoader.js";
import { DRACOLoader } from ".../DRACOLoader.js";
import { KTX2Loader } from ".../KTX2Loader.js";
import { MeshoptDecoder } from
".../libs/meshopt_decoder.module.js";
const draco = new
DRACOLoader().setDecoderPath("/draco/");
const ktx2 = new KTX2Loader()
  .setTranscoderPath("/basis/").detectSupport(renderer);
const loader = new GLTFLoader()
  .setDRACOLoader(draco).setKTX2Loader(ktx2)
  .setMeshoptDecoder(MeshoptDecoder);
const gltf = await loader.loadAsync("bookshelf.glb");
scene.add(gltf.scene);  // gltf.animations:
AnimationClip[]
```

Old to new three.js API

Books (r49-r69)	r187
`Geometry`, `Face3`, `CubeGeometry`	`BufferGeometry`, `BoxGeometry`
`ImageUtils.loadTexture`	`TextureLoader`
`Projector`	`Raycaster.setFromCamera`
`MeshFaceMaterial`	material array + groups
`PointCloud`	`Points`, `PointsMaterial`
`CanvasRenderer`, `Detector.js`	removed; WebGL 2 everywhere
JSON/CTM/AWD loaders	glTF 2.0 + `GLTFLoader`
`MorphAnimMesh`, `Animation`	`AnimationMixer`
`shadowMapEnabled`	`shadowMap.enabled`
`gammaOutput`	`outputColorSpace`
`examples/js` scripts	`three/addons/` modules
Physijs, dat.GUI	physics engines, lil-gui
`Clock`, `PostProcessing`	`Timer`, `RenderPipeline`

WebGPURenderer and TSL quick start

```
import * as THREE from "three/webgpu";
import { Fn, uniform, uv, time, sin, color }
  from "three/tsl";
const renderer = new
THREE.WebGPURenderer({antialias:true});
await renderer.init();   // WebGPU, else WebGL 2 fallback
const mat = new THREE.MeshStandardNodeMaterial();
mat.emissiveNode = color(0xffb347)

.mul(sin(time.mul(2).add(uv().x.mul(6))).mul(.5).add(.5))
;
// ShaderMaterial, onBeforeCompile, EffectComposer:
// not supported here; use TSL and RenderPipeline
```

WebGPU vs WebGL concept map

WebGL 2	WebGPU
context (global state)	adapter, device, queue
program, state toggles	render pipeline (state baked in)
uniforms, textures, samplers	bind groups
FBO	render pass with attachments
draw calls immediately	command encoder, submit
GLSL ES 3.00	WGSL
NDC z -1..1, UV origin bottom-left	NDC z 0..1, texture origin top-left
transform feedback	compute shaders

Browser	WebGL 2	WebGPU
Chrome, Edge	56 / 79	113 (Android 121)
Firefox	51	141 Windows
Safari, iOS	15	26